JAVA

QUESTION BANK ANSWERS

MODULE 1

1. Compare Procedure-Oriented Programming and Object-Oriented Programming

Definition

Procedure-Oriented Programming (POP): A programming approach in which a program is divided into functions or procedures that perform specific tasks.

Object-Oriented Programming (OOP): A programming approach in which a program is organized using objects that contain data and methods.

No.	Procedure-Oriented Programming (POP)	Object-Oriented Programming (OOP)
1	Focuses on functions or procedures.	Focuses on objects and classes.
2	Follows a top-down approach.	Generally follows a bottom-up approach.
3	Data and functions are separate.	Data and methods are combined into objects.
4	Provides less data security.	Supports data hiding through encapsulation.
5	Does not support inheritance.	Supports inheritance.
6	Does not directly support polymorphism.	Supports polymorphism.
7	Data may be shared among functions.	Data can be controlled using access modifiers.
8	Suitable for small and simple programs.	Suitable for large and complex programs.
9	Code reusability is comparatively limited.	Supports code reusability through inheritance and other features.
10	Example: C.	Examples: Java, C++, C#.

Example

POP example in C

#include <stdio.h>

```
void display() {
  printf("Hello World");
}

int main() {
  display();
  return 0;
}
```

Here, the program is divided into functions.

OOP example in Java

```
class Student {
  void display() {
    System.out.println("Hello World");
  }
}

class Main {
  public static void main(String[] args) {
    Student s = new Student();
    s.display();
  }
}
```

Here, the Student class contains a method, and an object calls that method.

Advantages of OOP

1. **Provides data security.**
2. **Supports code reusability.**
3. **Makes programs easier to maintain.**
4. **Supports inheritance and polymorphism.**

Disadvantages of OOP

1. **Requires understanding of classes and objects.**
2. **May require more memory.**

3. **Can be more complex for small programs.**

Conclusion: POP focuses on functions, whereas OOP focuses on objects. OOP is widely used for developing large and maintainable applications.

2. Compare C++ and Java Programming Based on Their Key Features

Definition

C++: A general-purpose programming language that supports procedural programming and object-oriented programming.

Java: A high-level, object-oriented programming language designed to support platform-independent execution through the Java Virtual Machine (JVM).

10 Differences between C++ and Java

No.	C++	Java
1	Developed by Bjarne Stroustrup.	Developed by James Gosling and his team at Sun Microsystems.
2	Supports procedural and object-oriented programming.	Primarily class-based and object-oriented.
3	Compiles into native machine code for a target platform.	Compiles into bytecode executed by the JVM.
4	Platform-dependent compiled executable.	Platform-independent bytecode when a compatible JVM is available.
5	Supports pointers and pointer arithmetic.	Uses references; no direct pointer arithmetic.
6	Supports multiple inheritance through classes.	Classes support single inheritance; multiple inheritance of type is supported through interfaces.
7	Supports operator overloading.	Does not support user-defined operator overloading.
8	Supports manual memory management and RAII.	Uses automatic garbage collection for objects.
9	Supports header files and preprocessor directives.	Uses packages and import statements.
10	Supports both constructors and destructors.	Supports constructors; no conventional destructors. Garbage collection manages object memory.

Example

C++

```
#include <iostream>

using namespace std;


int main() {

  cout << "Hello World";

  return 0;

}
```

Java

```
class Main {

  public static void main(String[] args) {

    System.out.println("Hello World");

  }

}
```

Advantages of Java over C++

1. **Platform-independent execution through JVM.**
2. **Automatic garbage collection.**
3. **No direct pointer arithmetic.**
4. **Built-in support for multithreading.**
5. **Strong runtime checking and exception handling.**

Conclusion

C++ provides more low-level control and native execution, while Java provides JVM-based portability, automatic memory management, and a managed runtime environment.

3. Apply the Principles of OOP to Explain Their Role in Java Programming

Definition

Object-Oriented Programming (OOP) is a programming methodology that organizes programs using classes and objects. It combines data and methods and provides features such as encapsulation, inheritance, polymorphism, and abstraction.

Java uses OOP principles to develop reusable, secure, and maintainable programs.

Four Main Principles of OOP

1. Encapsulation

Definition: Encapsulation is the process of combining data and methods into a single unit called a class and controlling access to the data.

- **Data members are declared using access modifiers.**
- **Private data is accessed through public methods.**
- **It provides data hiding and controlled access.**

Example:

```
class Student {
  private int age = 20;

  public int getAge() {
    return age;
  }
}
```

Here, age is private and can be accessed through the public getAge() method.

Role: Protects data from direct unauthorized access.

2. Inheritance

Definition: Inheritance is the process by which one class acquires the properties and methods of another class.

- **The existing class is called the parent or superclass.**
- **The new class is called the child or subclass.**
- **Java uses the extends keyword for class inheritance.**

Example:

```
class Animal {
  void eat() {
    System.out.println("Animal eats");
  }
}

class Dog extends Animal {
  void bark() {
    System.out.println("Dog barks");
  }
}
```

Role: Promotes code reusability and reduces duplication.

3. Polymorphism

Definition: Polymorphism means one name or interface can represent different behaviors.

Types:

1. **Compile-time polymorphism – method overloading.**
2. **Runtime polymorphism – method overriding.**

Example:

```
class Calculator {
  int add(int a, int b) {
    return a + b;
  }

  int add(int a, int b, int c) {
    return a + b + c;
  }
}
```

Here, the same method name add() performs different operations based on the arguments.

Role: Provides flexibility and allows methods to perform different tasks.

4. Abstraction

Definition: Abstraction is the process of hiding implementation details and showing only essential features to the user.

Java achieves abstraction using:

- **Abstract classes**
- **Interfaces**

Example:

```
abstract class Vehicle {
  abstract void start();
}

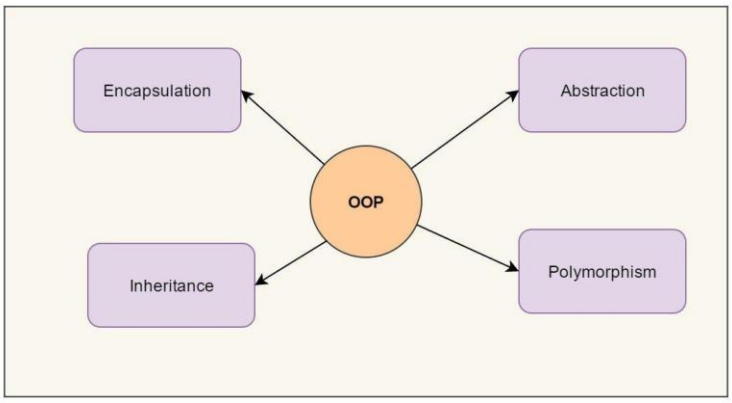
class Car extends Vehicle {
  void start() {
    System.out.println("Car starts");
  }
```

}

Here, the abstract class defines what the vehicle should do, while the child class provides the implementation.

Role: Reduces complexity and makes programs easier to understand.

Diagram: OOP Principles



Four Pillars of Object Oriented Programming

OOP Principles in Java

Encapsulation

Data hiding

Inheritance

Code reusability

Polymorphism

Many behaviors

Abstraction

Hides implementation

Advantages of OOP

1. **Code reusability.**
2. **Data security.**
3. **Easy maintenance.**
4. **Reduced code duplication.**
5. **Better program organization.**

Conclusion

The four principles of OOP make Java programs reusable, flexible, secure, and easier to maintain.

4. Identify the Key Features of Java Programming (Java Buzzwords)

Definition

Java is a high-level, class-based, object-oriented programming language. It is designed to be simple, secure, robust, portable, and suitable for developing different types of applications.

The important features of Java are called Java Buzzwords.

12 Key Features of Java

1. Simple

Java has a simple syntax similar to C and C++. It removes complicated features such as pointer arithmetic and supports automatic memory management.

2. Object-Oriented

Java uses classes and objects. It supports encapsulation, inheritance, polymorphism, and abstraction.

3. Platform Independent

Java programs are compiled into bytecode. The bytecode can run on different operating systems with a compatible JVM.

4. Portable

Java provides consistent behavior across platforms because of standardized data types and bytecode execution.

5. Secure

Java provides security features such as bytecode verification, access control, and runtime checks.

6. Robust

Java is robust because it supports:

- **Exception handling**
- **Automatic garbage collection**
- **Type checking**
- **Memory management**

7. Architecture Neutral

Java bytecode is not designed for a specific processor architecture. It can run on different hardware platforms using their JVM implementations.

8. Multithreaded

Java supports multiple threads running concurrently.

Example:

Thread t = new Thread();

t.start();

9. Distributed

Java supports network-based applications through libraries such as networking APIs.

10. High Performance

Java uses Just-In-Time (JIT) compilation to improve execution speed by compiling frequently executed bytecode into native machine code.

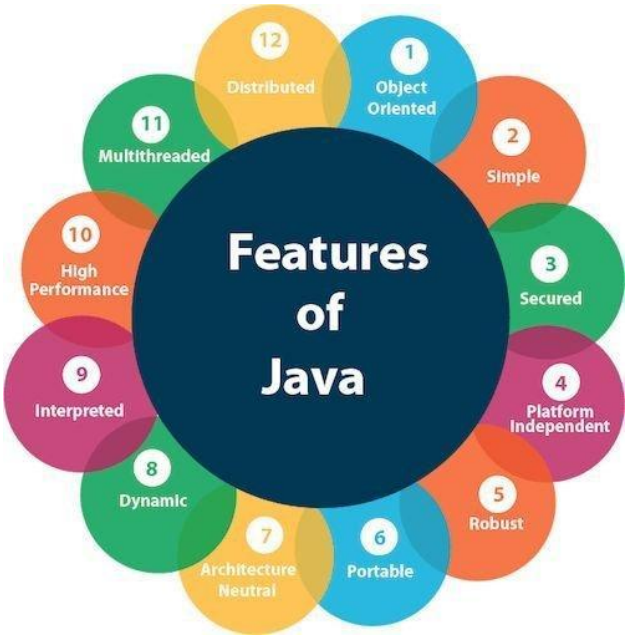
11. Dynamic

Java supports dynamic class loading and runtime linking. Classes can be loaded when they are needed.

12. Interpreted and Compiled

Java source code is compiled into bytecode. The JVM interprets or JIT-compiles the bytecode during execution.

Diagram: Java Features



Advantages

1. **Platform-independent execution.**
2. **Automatic memory management.**
3. **Built-in exception handling.**
4. **Supports multithreading.**
5. **Large standard library.**

Disadvantages

1. **May use more memory than some native programs.**

2. **Startup can be slower in some applications.**
3. **JVM is required to run Java bytecode.**
4. **Less direct hardware control than C or C++.**

Conclusion

Java buzzwords describe the important characteristics that make Java a portable, secure, robust, and widely used programming language.

5. Define Bytecode. Explain the Steps to Build and Run a Simple "Hello World" Program in Java

Definition of Bytecode

Bytecode is an intermediate, platform-independent instruction format generated by the Java compiler from Java source code.

It is stored in a .class file and executed by the Java Virtual Machine (JVM).

Steps to Build and Run a Java Program

Step 1: Write the Java program

Create a file named HelloWorld.java.

```
class HelloWorld {
  public static void main(String[] args) {
    System.out.println("Hello World");
  }
}
```

Step 2: Compile the program

Open the terminal or command prompt and execute:

```
javac HelloWorld.java
```

The Java compiler converts source code into bytecode.

Output file:

HelloWorld.class

Step 3: Run the program

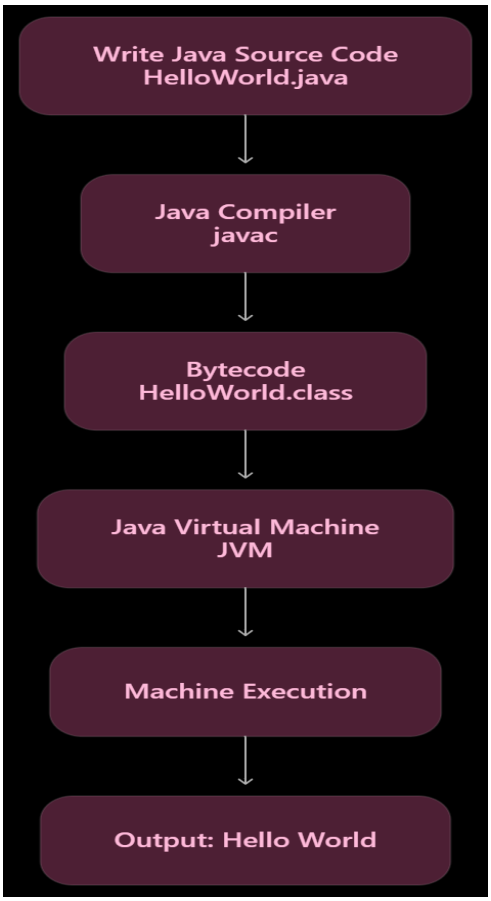
Execute:

```
java HelloWorld
```

Step 4: Observe the output

```
Hello World
```

Flowchart: Java Program Execution



Explanation of Important Terms

Term	Meaning
.java	Java source code file
javac	Java compiler
.class	Bytecode file
JVM	Executes Java bytecode
java	Command used to launch a Java program

Advantages of Bytecode

1. **Platform-independent.**
2. **Supports Java portability.**
3. **Provides bytecode verification.**
4. **Allows JVM optimization.**
5. **Separates source code from machine-specific execution.**

Conclusion

Bytecode is an intermediate representation of Java code. The Java compiler converts source code into bytecode, and the JVM executes it to produce the output.

6. Define Variable. Identify Primitive and Non-Primitive Data Types with Examples

Definition of Variable

A variable is a named memory location used to store data that can be accessed and modified during program execution.

In Java, every variable must have a data type that specifies the kind of value it can store.

Syntax

dataType variableName = value;

Example:

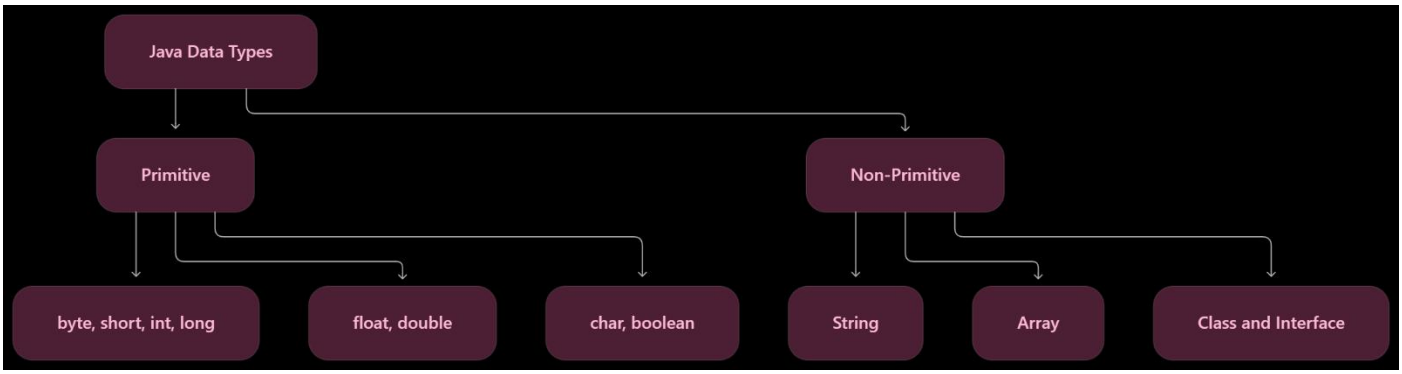
int age = 20;

Here:

- **int – data type**
- **age – variable name**
- **20 – value stored in the variable**

Types of Data Types in Java

Java data types are classified into two main categories:



A. Primitive Data Types

Primitive data types are predefined data types that directly store simple values.

Java has 8 primitive data types.

No.	Data Type	Size	Example
1	byte	8 bits	byte a = 10;

2	short	16 bits	short b = 100;
3	int	32 bits	int age = 20;
4	long	64 bits	long n = 1000L;
5	float	32 bits	float x = 10.5f;
6	double	64 bits	double y = 25.75;
7	char	16 bits	char c = 'A';
8	boolean	JVM-dependent storage representation	boolean flag = true;

Explanation

1. byte: Stores small integer values.

byte age = 20;

2. short: Stores integer values larger than the typical byte range.

short number = 1000;

3. int: Commonly used for whole numbers.

int marks = 85;

4. long: Used for larger integer values.

long population = 100000L;

5. float: Stores decimal values with single precision.

float price = 12.5f;

6. double: Stores decimal values with double precision.

double salary = 25000.75;

7. char: Stores a single UTF-16 code unit.

char grade = 'A';

8. boolean: Stores either true or false.

boolean result = true;

B. Non-Primitive Data Types

Non-primitive data types are also called reference types. They refer to objects or arrays rather than directly storing primitive values.

1. String

Used to store a sequence of characters.

String name = "Java";

2. Array

Used to store multiple values of the same declared type.



```
int[] marks = {80, 90, 85};
```

3. Class

A class is a blueprint for creating objects.

```
class Student {
  int age = 20;
}
```

4. Interface

An interface defines a contract that implementing classes must follow.

```
interface Animal {
  void sound();
}
```

Complete Java Example

```
class Main {
  public static void main(String[] args) {

    // Primitive data types
    int age = 20;
    double marks = 85.5;
    char grade = 'A';
    boolean pass = true;

    // Non-primitive data types
    String name = "Java";
    int[] numbers = {10, 20, 30};

    System.out.println(age);
    System.out.println(marks);
    System.out.println(grade);
    System.out.println(pass);
    System.out.println(name);
    System.out.println(numbers[0]);
  }
```

}

Output

20

85.5

A

true

Java

10

Advantages

1. **Variables allow data to be stored and reused.**
2. **Data types provide type checking.**
3. **Primitive types are efficient for simple values.**
4. **Reference types support objects and complex data structures.**

Conclusion

Java provides eight primitive data types and several reference types, including strings, arrays, classes, and interfaces. These data types help store and organize different kinds of information.

7. What is Type Conversion? Explain the Different Types of Type Conversion with Suitable Examples

Definition

Type conversion is the process of converting a value from one data type into another data type.

In Java, type conversion is mainly classified into:

1. **Implicit type conversion (Widening)**
2. **Explicit type conversion (Narrowing)**

1. Implicit Type Conversion (Widening)

Definition: Implicit type conversion is the automatic conversion of a smaller compatible data type into a larger data type.

It is generally performed by the Java compiler without requiring explicit casting.

Example

```
class Main {
  public static void main(String[] args) {

    int a = 10;
```

```
    double b = a;


    System.out.println(b);
  }
}
```

Output

10.0

Here, the integer value 10 is automatically converted into the double value 10.0.

Widening Order

```
flowchart LR
  A["byte"] --> B["short"]
  B --> C["int"]
  C --> D["long"]
  D --> E["float"]
  E --> F["double"]
  G["char"] --> C
```

Note: This diagram shows common widening paths, not every possible conversion. Java also permits some direct conversions, such as int to double.

Advantages

1. **Automatic conversion.**
2. **Simple to use.**
3. **Generally avoids loss of integer magnitude for the standard widening paths.**
4. **Reduces explicit casting.**

2. Explicit Type Conversion (Narrowing)

Definition: Explicit type conversion is the conversion of a larger data type into a smaller data type using a cast operator.

The programmer must specify the target type.

Syntax

```
targetType variable = (targetType) value;
```

Example

```
class Main {
  public static void main(String[] args) {
```

```
    double a = 10.75;
    int b = (int) a;

    System.out.println(b);
  }
}
```

Output

10

Here, the decimal portion is removed when converting the value to int.

Important: Narrowing may cause loss of information or overflow.

Advantages

1. **Allows conversion when automatic conversion is not permitted.**
2. **Gives the programmer control over the target type.**
3. **Useful when a particular data type is required.**

Disadvantages

1. **May lose decimal values.**
2. **May cause overflow.**
3. **Requires careful handling.**

3. String Conversion

Java also supports conversion between strings and other types using methods such as String.valueOf() and parsing methods.

Number to String

```
int a = 100;

String s = String.valueOf(a);

System.out.println(s);
```

Output:

100

String to Number

```
String s = "100";
```

int a = Integer.parseInt(s);

System.out.println(a);

Output:

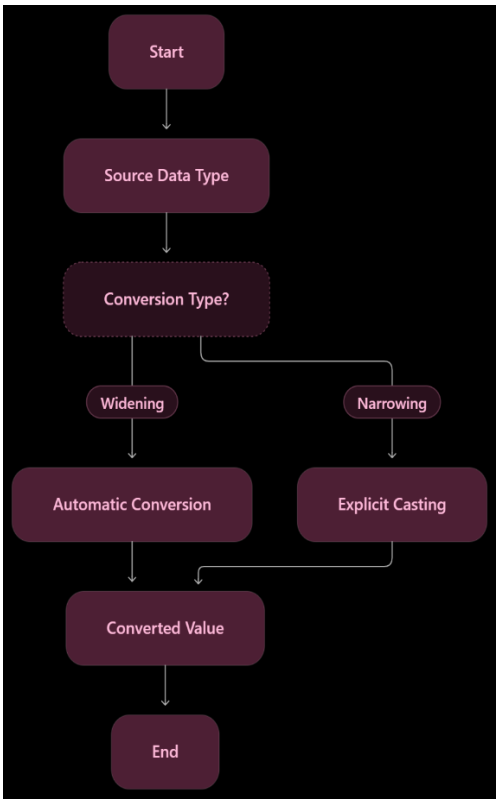
100

Note: Parsing an invalid numeric string may cause NumberFormatException.

Difference Between Widening and Narrowing

No.	Widening	Narrowing
1	Smaller type to larger compatible type.	Larger type to smaller type.
2	Usually automatic.	Requires explicit casting.
3	No cast operator required.	Cast operator is required.
4	Generally preserves integer value.	May lose information.
5	Example: int to double.	Example: double to int.

Flowchart: Type Conversion



1. Define Array. Explain its Types, Declaration and Initialization in Java

Definition

An array is a collection of elements of the same data type stored under a single variable name. Each element is accessed using an index.

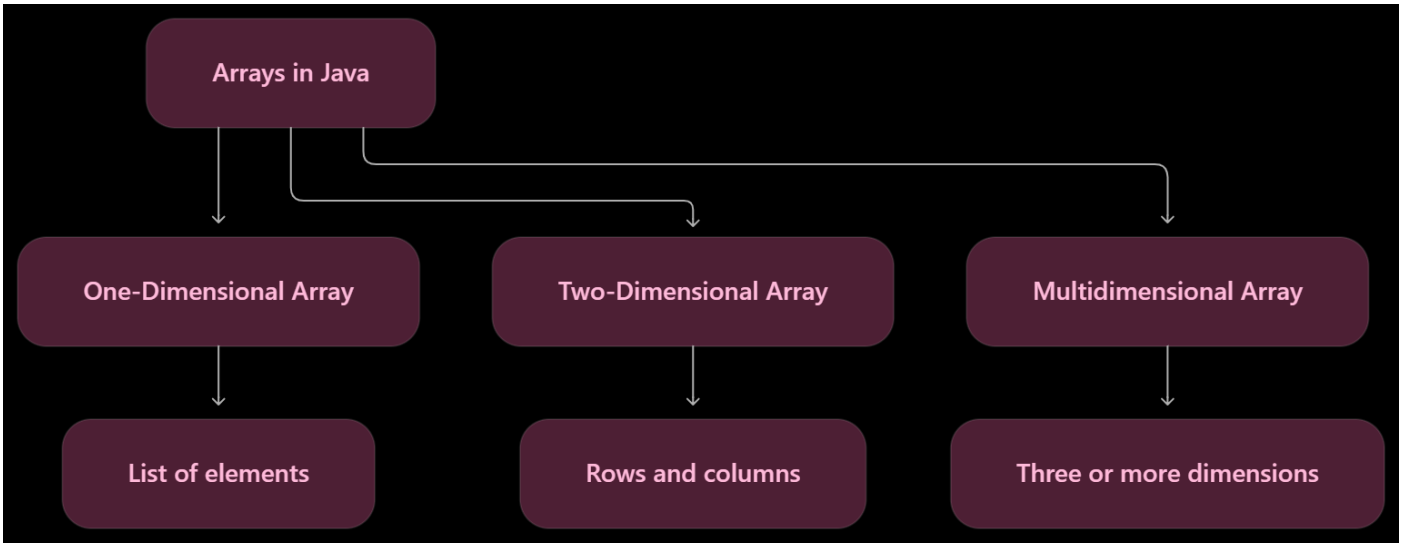
In Java, array indexing starts from 0.

Basic Concepts

- **An array stores multiple values of the same declared type.**
- **Array elements are accessed using index numbers.**
- **Arrays have a fixed length after creation.**
- **Java arrays are objects.**
- **The length property gives the number of elements.**

Types of Arrays in Java

Java arrays are commonly classified into three types:



1. One-Dimensional Array

A one-dimensional array stores elements in a single sequence.

Declaration:

```
int[] a;
```

Initialization:

```
int[] a = {10, 20, 30, 40};
```

Accessing elements:

```
System.out.println(a[0]);
```

Output:

10

2. Two-Dimensional Array

A two-dimensional array stores elements in rows and columns. It is useful for representing matrices and tables.

Declaration:

```
int[][] a;
```

Initialization:

```
int[][] a = {
  {1, 2},
  {3, 4}
};
```

Accessing an element:

```
System.out.println(a[0][1]);
```

Output:

2

3. Multidimensional Array

A multidimensional array contains two or more dimensions.

Example: Three-dimensional array.

```
int[][][] a = new int[2][3][4];
```

It can be used to represent data involving multiple levels, such as groups, rows, and columns.

Array Declaration and Initialization

Method 1: Declaration and initialization together

```
int[] marks = {80, 90, 85};
```

Method 2: Declaration and creation separately

```
int[] marks;

marks = new int[3];
```

Method 3: Assigning values individually

```
int[] marks = new int[3];

marks[0] = 80;
marks[1] = 90;
marks[2] = 85;
```

Example Program

```
class Main {
```

```
  public static void main(String[] args) {

    int[] marks = {80, 90, 85};

    System.out.println("Array elements:");

    for (int i = 0; i < marks.length; i++) {
      System.out.println(marks[i]);
    }
  }
}
```

Output

Array elements:

80

90

85

Advantages

1. **Stores multiple values using one variable.**
2. **Easy access using index.**
3. **Reduces the number of variables.**
4. **Useful for sorting and searching.**
5. **Supports matrix operations.**

Disadvantages

1. **Fixed length after creation.**
2. **Stores elements of one declared type.**
3. **Insertion and deletion may require shifting elements.**
4. **Invalid indexes cause ArrayIndexOutOfBoundsException.**

Conclusion

Arrays are important in Java for storing and processing multiple values efficiently. One-dimensional, two-dimensional, and multidimensional arrays are used according to the structure of the data.

2. What are Iterative Statements? Explain Any One with a Suitable Example

Definition

ACE

Iterative statements are statements used to execute a block of code repeatedly until a specified condition becomes false.

They are also called looping statements.

Types of Iterative Statements in Java

Java provides three basic looping statements:

1. **for loop**
2. **while loop**
3. **do-while loop**

For Loop

Definition

The for loop is used to execute a block of statements repeatedly. It is generally suitable when the number of iterations is known.

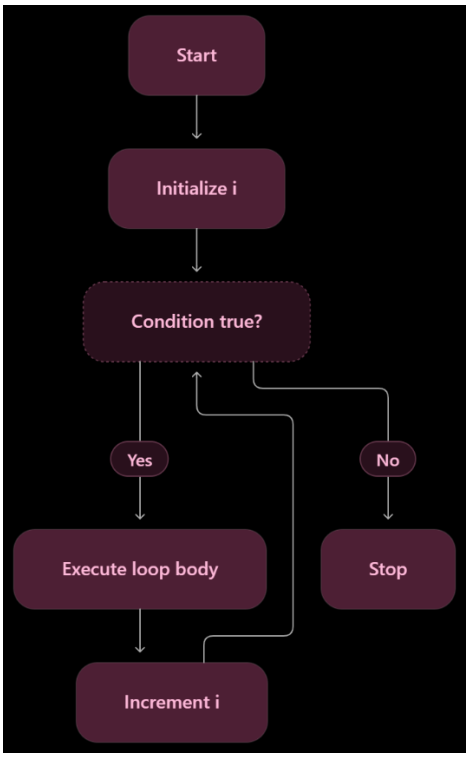
Syntax

```
for (initialization; condition; increment) {
  // Statements
}
```

Working Steps

1. **Initialization is performed.**
2. **The condition is checked.**
3. **If the condition is true, the loop body executes.**
4. **The variable is updated.**
5. **The condition is checked again.**
6. **The loop stops when the condition becomes false.**

Flowchart



Example Program

Print numbers from 1 to 5.

```
class Main {
  public static void main(String[] args) {

    for (int i = 1; i <= 5; i++) {
      System.out.println(i);
    }
  }
}
```

Output

1

2

3

4

5

Explanation

- **int i = 1 initializes the loop.**
- **i <= 5 checks the condition.**
- **System.out.println(i) prints the number.**

- **i++ increases the value by one.**
- **When i becomes 6, the condition is false and the loop terminates.**

Advantages of Iterative Statements

1. **Reduce repeated code.**
2. **Save programming time.**
3. **Improve readability.**
4. **Useful for processing arrays.**
5. **Perform repeated calculations efficiently.**

Disadvantages

1. **Incorrect conditions may cause infinite loops.**
2. **Improper initialization may produce incorrect results.**
3. **Nested loops can increase execution time.**

3. Explain Jump Statements in Java with Suitable Examples

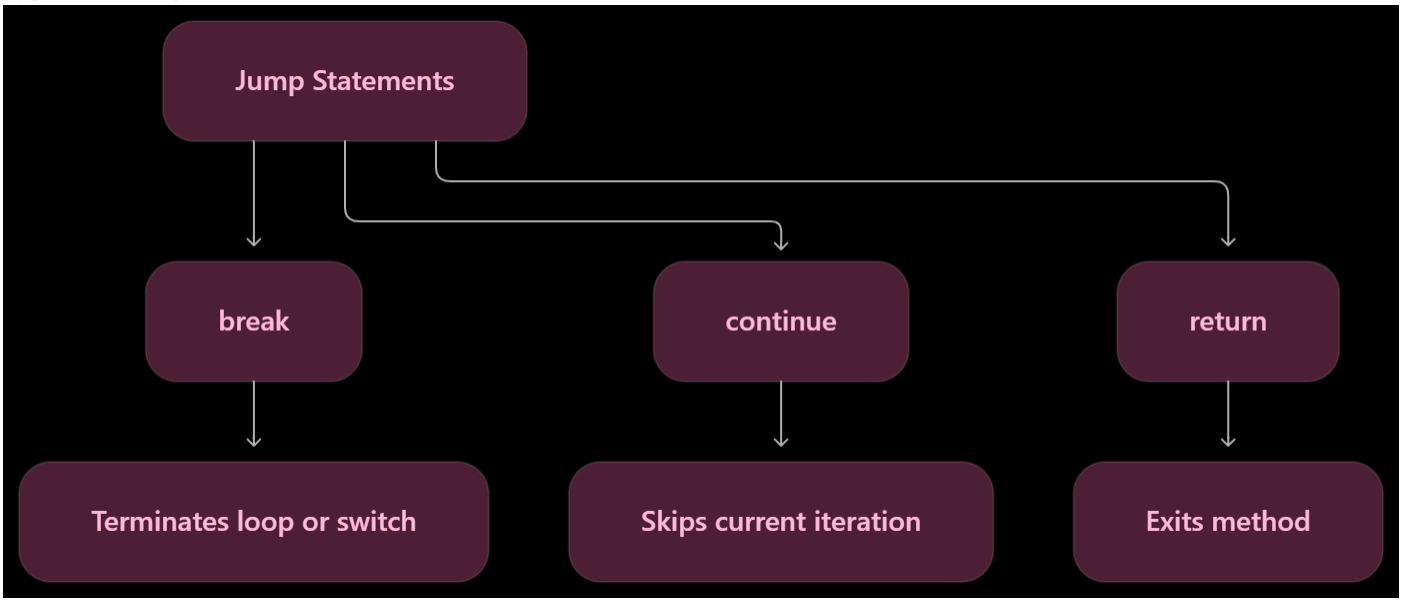
Definition

Jump statements are statements that transfer control from one part of a program to another part.

Java provides three main jump statements:

1. **break**
2. **continue**
3. **return**

Types of Jump Statements



1. break Statement

Definition

The break statement terminates the nearest enclosing loop or switch statement.

Example

```
class Main {
  public static void main(String[] args) {

    for (int i = 1; i <= 5; i++) {

      if (i == 3) {
        break;
      }

      System.out.println(i);
    }
  }
}
```

Output

1

2

Explanation

When i becomes 3, the break statement terminates the loop immediately.

2. continue Statement

Definition

The continue statement skips the remaining statements in the current loop iteration and proceeds to the next iteration.

Example

```
class Main {
  public static void main(String[] args) {

    for (int i = 1; i <= 5; i++) {

      if (i == 3) {
```

ACE

```
            continue;
        }

        System.out.println(i);
      }
    }
}
```

Output

1

2

4

5

Explanation

When i becomes 3, the print statement is skipped. The loop continues with the next iteration.

3. return Statement

Definition

The return statement terminates the execution of a method and optionally returns a value to the calling method.

Example

```
class Main {

  static int add() {
    return 10 + 20;
  }

  public static void main(String[] args) {

    int result = add();

    System.out.println(result);
  }
}
```

Output

30

Explanation

The return statement returns the value 30 from the add() method to the calling method.

Advantages

1. **Provides control over program execution.**
2. **Helps terminate loops when required.**
3. **Avoids unnecessary iterations.**
4. **Allows methods to return results.**

Disadvantages

1. **Excessive use may make program flow difficult to understand.**
2. **Incorrect placement can terminate loops or methods unexpectedly.**
3. **May reduce readability if used without proper structure.**

4. What are Conditional Statements in Java? Explain Different Types with Examples

Definition

Conditional statements are decision-making statements that execute different blocks of code depending on whether a specified condition is true or false.

They control the flow of execution in Java programs.

Types of Conditional Statements

Java provides the following conditional statements:

1. **if statement**
2. **if-else statement**
3. **else-if ladder**
4. **Nested if statement**
5. **switch statement**

1. if Statement

Definition

The if statement executes a block of code only when the given condition is true.

General Form

```
if (condition) {
  // Statements
}
```

Example

```
class Main {
  public static void main(String[] args) {

    int age = 20;

    if (age >= 18) {
      System.out.println("Eligible");
    }
  }
}
```

Output:

```
Eligible
```

2. if-else Statement

Definition

The if-else statement executes one block when the condition is true and another block when it is false.

General Form

```
if (condition) {
  // True block
} else {
  // False block
}
```

Example

```
class Main {
  public static void main(String[] args) {

    int marks = 35;
```

```
    if (marks >= 40) {
      System.out.println("Pass");
    } else {
      System.out.println("Fail");
    }
  }
}
```

Output:

Fail

3. else-if Ladder

Definition

The else-if ladder is used to check multiple conditions one after another.

The first true condition's block is executed.

General Form

```
if (condition1) {
  // Statements
} else if (condition2) {
  // Statements
} else if (condition3) {
  // Statements
} else {
  // Default statements
}
```

Example

```
class Main {
  public static void main(String[] args) {

    int marks = 85;

    if (marks >= 90) {
      System.out.println("A Grade");
```

```
    } else if (marks >= 75) {
      System.out.println("B Grade");
    } else if (marks >= 50) {
      System.out.println("C Grade");
    } else {
      System.out.println("Fail");
    }
  }
}
```

Output:

B Grade

4. Nested if Statement

Definition

A nested if statement is an if statement placed inside another if statement.

It is used when a decision depends on another decision.

General Form

```
if (condition1) {
  if (condition2) {
    // Statements
  }
}
```

Example

```
class Main {
  public static void main(String[] args) {

    int age = 20;
    boolean hasID = true;

    if (age >= 18) {

      if (hasID) {
        System.out.println("Entry allowed");
```

```
        }
      }
    }
}
```

Output:

Entry allowed

5. switch Statement

Definition

The switch statement selects one block of code from multiple alternatives based on the value of an expression.

General Form

```
switch (expression) {

  case value1:
    // Statements
    break;

  case value2:
    // Statements
    break;

  default:
    // Default statements
}
```

Example

```
class Main {
  public static void main(String[] args) {

    int day = 2;

    switch (day) {

      case 1:
```

```
            System.out.println("Monday");
            break;

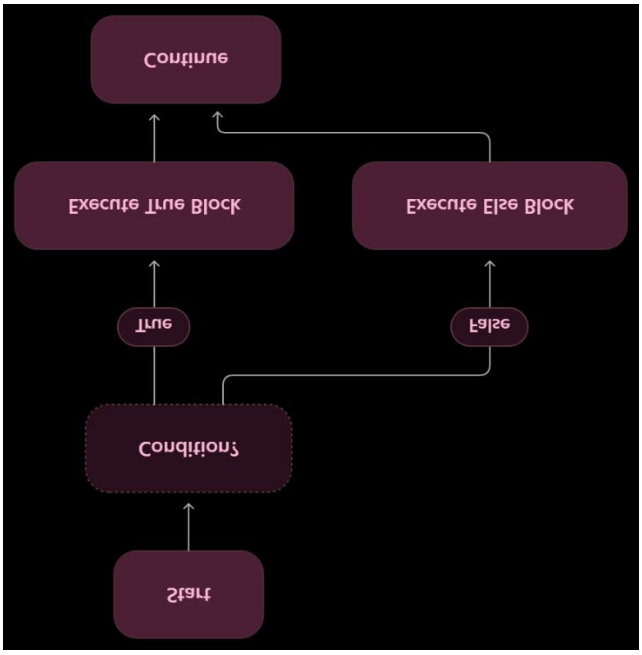
        case 2:
            System.out.println("Tuesday");
            break;

        default:
            System.out.println("Invalid day");
    }
  }
}
```

Output:

Tuesday

Flowchart



Advantages

1. **Supports decision-making.**
2. **Controls program execution.**
3. **Makes programs flexible.**
4. **Helps implement complex logic.**

Disadvantages

1. **Too many conditions may reduce readability.**
2. **Incorrect conditions may produce wrong results.**
3. **Nested conditions may become difficult to maintain.**

Conclusion

Conditional statements are essential for decision-making in Java. They allow programs to execute different instructions based on conditions.

5. Define Operator. Identify the Different Types of Operators in Java

Definition

An operator is a special symbol used to perform operations on variables and values.

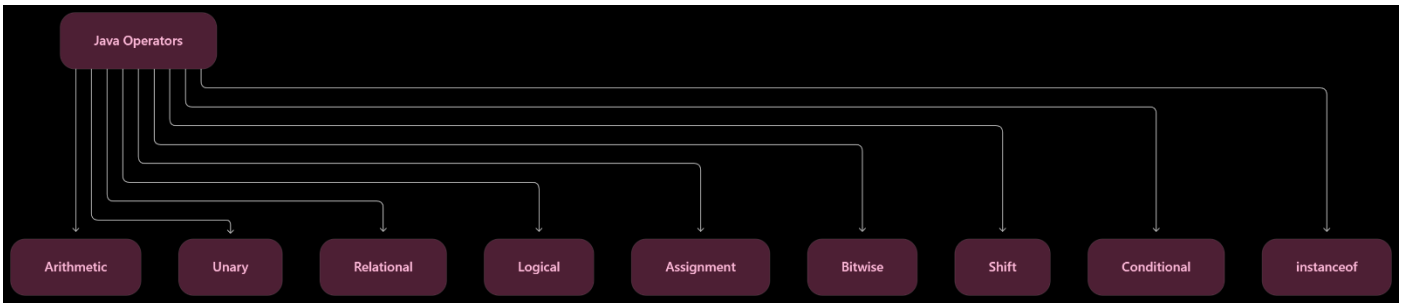
Example:

int c = a + b;

Here, + is an operator that performs addition.

Types of Operators in Java

Java operators are classified into the following categories:



1. Arithmetic Operators

Used to perform mathematical operations.

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus

Example:

int a = 10, b = 3;

System.out.println(a + b);

System.out.println(a % b);

Output:

13

1

2. Unary Operators

Operate on a single operand.

Operator	Meaning
+	Unary plus
-	Unary minus
++	Increment
--	Decrement
!	Logical NOT
~	Bitwise complement

Example:

int a = 5;

a++;

System.out.println(a);

Output:

6

3. Relational Operators

Used to compare two values.

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to

<= Less than or equal to

Example:

```
int a = 10, b = 20;


System.out.println(a < b);
```

Output:

true

4. Logical Operators

Used to combine or reverse Boolean conditions.

Operator	Meaning
&&	Logical AND
||	Logical OR
!	Logical NOT

Example:

```
int age = 20;


System.out.println(age >= 18 && age <= 60);
```

Output:

true

5. Assignment Operators

Used to assign values to variables.

Operator	Example
=	a = 10
+=	a += 5
-=	a -= 5
*=	a *= 5
/=	a /= 5
%=	a %= 5

Example:

```
int a = 10;


a += 5;
```

System.out.println(a);

Output:

15

6. Bitwise Operators

Perform operations on individual bits of integer values.

| Operator | Meaning |
|---|---|
| & | Bitwise AND |
| \| | Bitwise OR |
| ^ | Bitwise XOR |
| ~ | Bitwise complement |

Example:

int a = 5, b = 3;

System.out.println(a & b);

Output:

1

7. Shift Operators

Used to shift bits left or right.

Operator	Meaning
<<	Left shift
>>	Signed right shift
>>>	Unsigned right shift

Example:

int a = 4;

System.out.println(a << 1);

Output:

8

8. Conditional Operator

The conditional operator ?: is a shorthand for a simple if-else expression.

Syntax:

condition ? value1 : value2;

Example:

```
int a = 10;


String result = (a >= 18) ? "Adult" : "Minor";


System.out.println(result);
```

Output:

Minor

9. instanceof Operator

Used to check whether an object is an instance of a particular class or type.

Example:

```
String name = "Java";


System.out.println(name instanceof String);
```

Output:

true

Advantages

1. **Simplifies calculations.**
2. **Supports decision-making.**
3. **Helps manipulate data.**
4. **Makes programs efficient.**

Conclusion

Java operators are used for arithmetic calculations, comparisons, logical operations, assignments, bit manipulation, and decision-making.

6. What are Access Specifiers in Java? Explain Different Types

Definition

Access specifiers are keywords used to control the visibility and accessibility of classes, variables, methods, and constructors in Java.

They help implement data hiding and encapsulation.

Types of Access Specifiers

Java provides four access levels:

1. **private**
2. **default**
3. **protected**
4. **public**

1. private

Definition: The private access specifier allows access only within the same class.

Example:

```
class Student {

  private int age = 20;

  void display() {
    System.out.println(age);
  }
}
```

Here, age can be directly accessed only inside the Student class.

Use: Provides strong data hiding.

2. default

Definition: When no access specifier is specified, Java uses default access. It allows access within the same package.

Example:

```
class Student {

  int age = 20;
```

```
  void display() {
    System.out.println(age);
  }
}
```

Here, age and display() have default access.

Use: Allows package-level access without using a keyword.

3. protected

Definition: The protected access specifier allows access within the same package and also through inheritance in subclasses in other packages, subject to Java's protected-access rules.

Example:

```
class Student {

  protected int age = 20;
}
```

Use: Supports inheritance and controlled access.

4. public

Definition: The public access specifier allows access from any class, subject to the visibility of the containing class and module.

Example:

```
public class Student {

  public int age = 20;
}
```

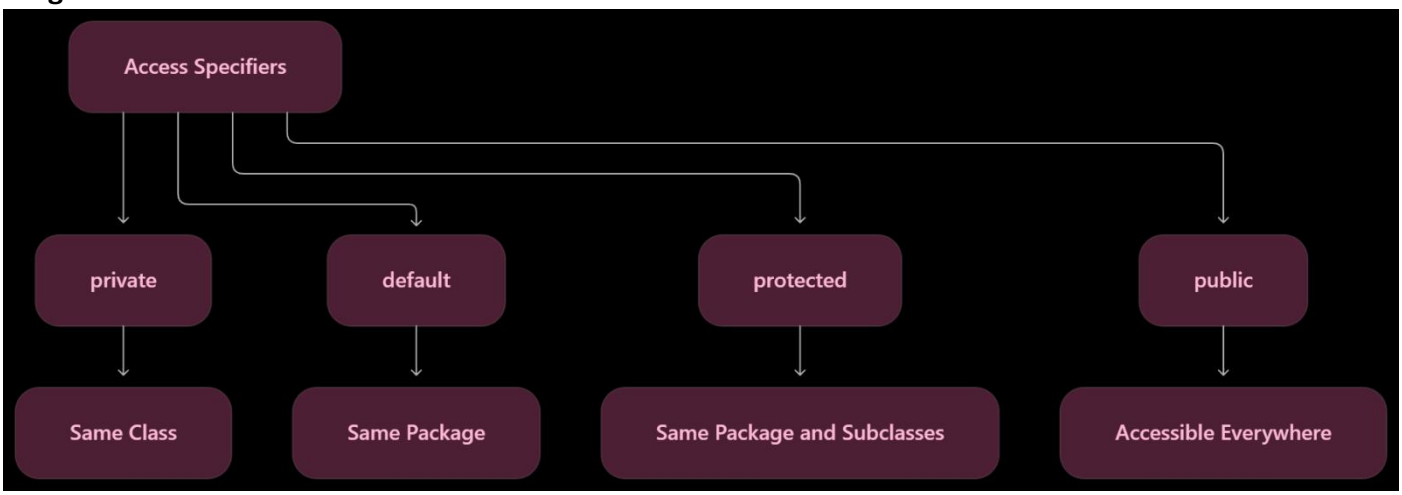
Use: Provides widely accessible methods and classes.

Comparison Table

Access Specifier	Same Class	Same Package	Subclass in Other Package	Other Package
private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes*	No
public	Yes	Yes	Yes	Yes

***Protected access in another package is available through inheritance and follows Java's subclass access rules.**

Diagram



Advantages

1. **Provides data security.**
2. **Supports encapsulation.**
3. **Controls access to class members.**
4. **Improves program organization.**
5. **Helps maintain code.**

Conclusion

Access specifiers control the visibility of Java class members. They provide different levels of access and help protect data through encapsulation.

7. Develop a Java Program to Add Two Matrices of Suitable Order N

Question: Develop a Java program to add two matrices of suitable order N. The value of N should be read from command-line arguments.

Definition

Matrix addition is the process of adding corresponding elements of two matrices having the same number of rows and columns.

If A and B are two matrices of order N × N, then:

\[C_{ij}=A_{ij}+B_{ij} \]

Where:

- **A = First matrix**
- **B = Second matrix**
- **C = Resultant matrix**

Algorithm

1. **Start.**
2. **Read the matrix order N from command-line arguments.**
3. **Create three matrices A, B, and C of size N × N.**
4. **Read elements of matrix A.**
5. **Read elements of matrix B.**
6. **Add corresponding elements of A and B.**
7. **Store the result in matrix C.**
8. **Display matrix C.**
9. **Stop.**

Java Program

```
import java.util.Scanner;

class MatrixAddition {

  public static void main(String[] args) {

    // Read matrix order from command line
    if (args.length != 1) {
      System.out.println("Usage: java MatrixAddition N");
      return;
    }

    int n = Integer.parseInt(args[0]);

    if (n <= 0) {
      System.out.println("N must be positive");
      return;
    }

    Scanner sc = new Scanner(System.in);

    int[][] a = new int[n][n];
```

```
int[][] b = new int[n][n];
int[][] c = new int[n][n];

// Read first matrix
System.out.println("Enter first matrix:");

for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
    a[i][j] = sc.nextInt();
  }
}

// Read second matrix
System.out.println("Enter second matrix:");

for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
    b[i][j] = sc.nextInt();
  }
}

// Add matrices
for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
    c[i][j] = a[i][j] + b[i][j];
  }
}

// Display result
System.out.println("Resultant Matrix:");

for (int i = 0; i < n; i++) {
```



```
        for (int j = 0; j < n; j++) {
          System.out.print(c[i][j] + " ");
        }

        System.out.println();
      }

      sc.close();
    }
}
```

Execution Steps

Step 1: Save the file

Save the program as:

MatrixAddition.java

Step 2: Compile

javac MatrixAddition.java

Step 3: Run with command-line argument

java MatrixAddition 2

Here, 2 represents the order of the matrix.

Step 4: Enter the first matrix

1 2

3 4

Step 5: Enter the second matrix

5 6

7 8

Output

Resultant Matrix:

6 8

10 12

Explanation

For the first matrix:

\[A=\begin{bmatrix}1&2\\3&4\end{bmatrix} \]

For the second matrix:

\[B=\begin{bmatrix}5&6\\7&8\end{bmatrix} \]

Addition:

\[C=A+B=\begin{bmatrix}1+5&2+6\\3+7&4+8\end{bmatrix} \]

Result:

\[C=\begin{bmatrix}6&8\\10&12\end{bmatrix} \]

Advantages

1. **Uses two-dimensional arrays.**
2. **Demonstrates nested loops.**
3. **Supports matrix calculations.**
4. **Uses command-line arguments.**
5. **Uses Scanner for input.**

Conclusion

Matrix addition in Java is performed by adding corresponding elements of two matrices using nested loops. The order N is obtained from the command line, and the resultant matrix is displayed.

8. Develop a Java Program to Iterate over Arrays Using for-each Loop to Compute Average of N Natural Numbers Using Scanner

Definition

The for-each loop is an enhanced loop in Java used to access and process each element of an array or collection without manually using an index.

The average is calculated by dividing the sum of all elements by the number of elements.

Formula

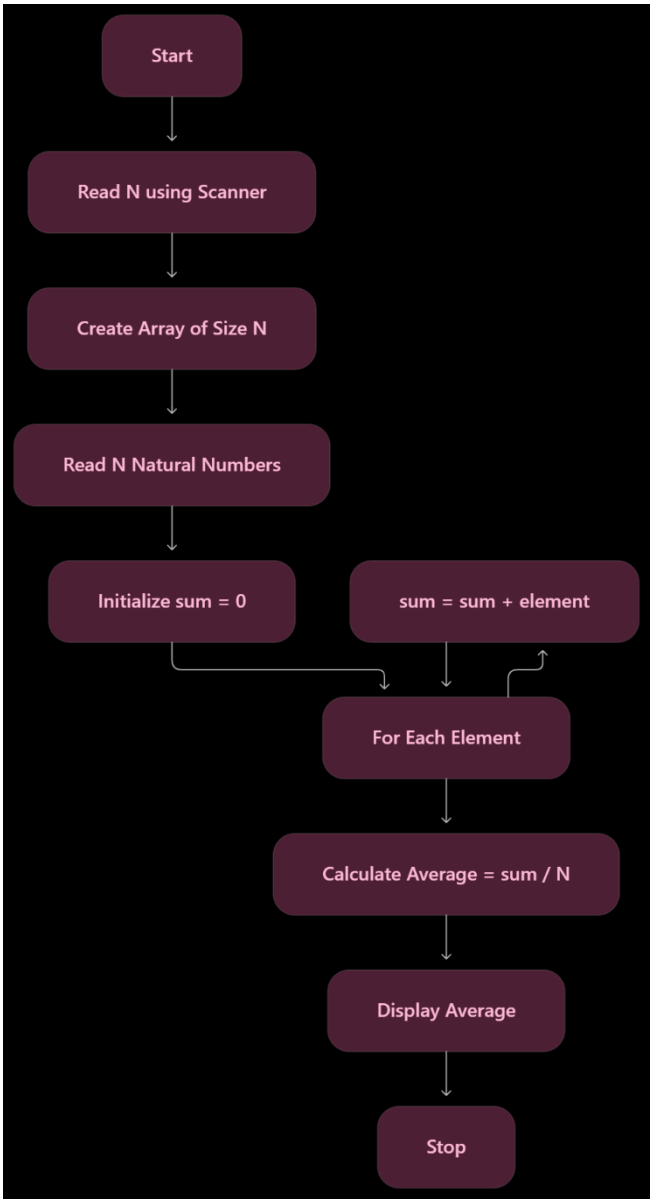
\[\text{Average}=\frac{\text{Sum of elements}}{N} \]

Algorithm

1. **Start.**
2. **Create a Scanner object.**
3. **Read the number N of natural numbers.**
4. **Create an integer array of size N.**
5. **Read N natural numbers into the array.**
6. **Initialize sum to zero.**
7. **Use a for-each loop to add all array elements.**

8. **Calculate average as sum divided by N.**
9. **Display the sum and average.**
10. **Stop.**

Flowchart



Java Program

```
import java.util.Scanner;

class AverageArray {

  public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);
```

```
System.out.print("Enter N: ");
int n = sc.nextInt();

if (n <= 0) {
   System.out.println("N must be positive");
   sc.close();
   return;
}

int[] numbers = new int[n];

System.out.println("Enter " + n + " natural numbers:");

// Read array elements
for (int i = 0; i < n; i++) {

   numbers[i] = sc.nextInt();

   if (numbers[i] <= 0) {
      System.out.println(
         "Enter only natural numbers"
      );

      sc.close();
      return;
   }
}

int sum = 0;

// Iterate using for-each loop
```

```
    for (int num : numbers) {
      sum = sum + num;
    }

    // Calculate average
    double average = (double) sum / n;

    System.out.println("Sum = " + sum);
    System.out.println("Average = " + average);

    sc.close();
  }
}
```

Example Input

Enter N: 5

Enter 5 natural numbers:

1

2

3

4

5

Output

Sum = 15

Average = 3.0

Explanation

- **Scanner reads input from the keyboard.**
- **int[] numbers stores the natural numbers.**
- **The first for loop reads the array elements.**
- **The for-each loop accesses each element using num.**
- **The sum is calculated by adding every element.**
- **The average is calculated by dividing the sum by N.**

For-each Loop Syntax

```
for (dataType variable : array) {
   // Statements
}
```

Example:

```
for (int num : numbers) {
   sum += num;
}
```

Here, num receives each array element one by one.

Advantages

1. **Simple and readable syntax.**
2. **No need to use array indexes.**
3. **Reduces coding errors.**
4. **Useful for calculating sum and average.**
5. **Easy to process arrays.**

Disadvantages

1. **Does not directly provide the index.**
2. **Not suitable for modifying array elements through the loop variable.**
3. **Cannot directly traverse an array backward.**

MODULE 2

1. Define Class and Object. Explain Class Definition, Object Creation and Accessing Members

Definition

Class: A class is a blueprint or template used to create objects. It contains data members (variables) and methods (functions).

Object: An object is an instance of a class. It contains state represented by data and behavior represented by methods.

General Form of Class Definition

```
class ClassName {

   // Data members
   dataType variableName;
```

```
    // Method
    returnType methodName() {
        // Statements
    }
}
```

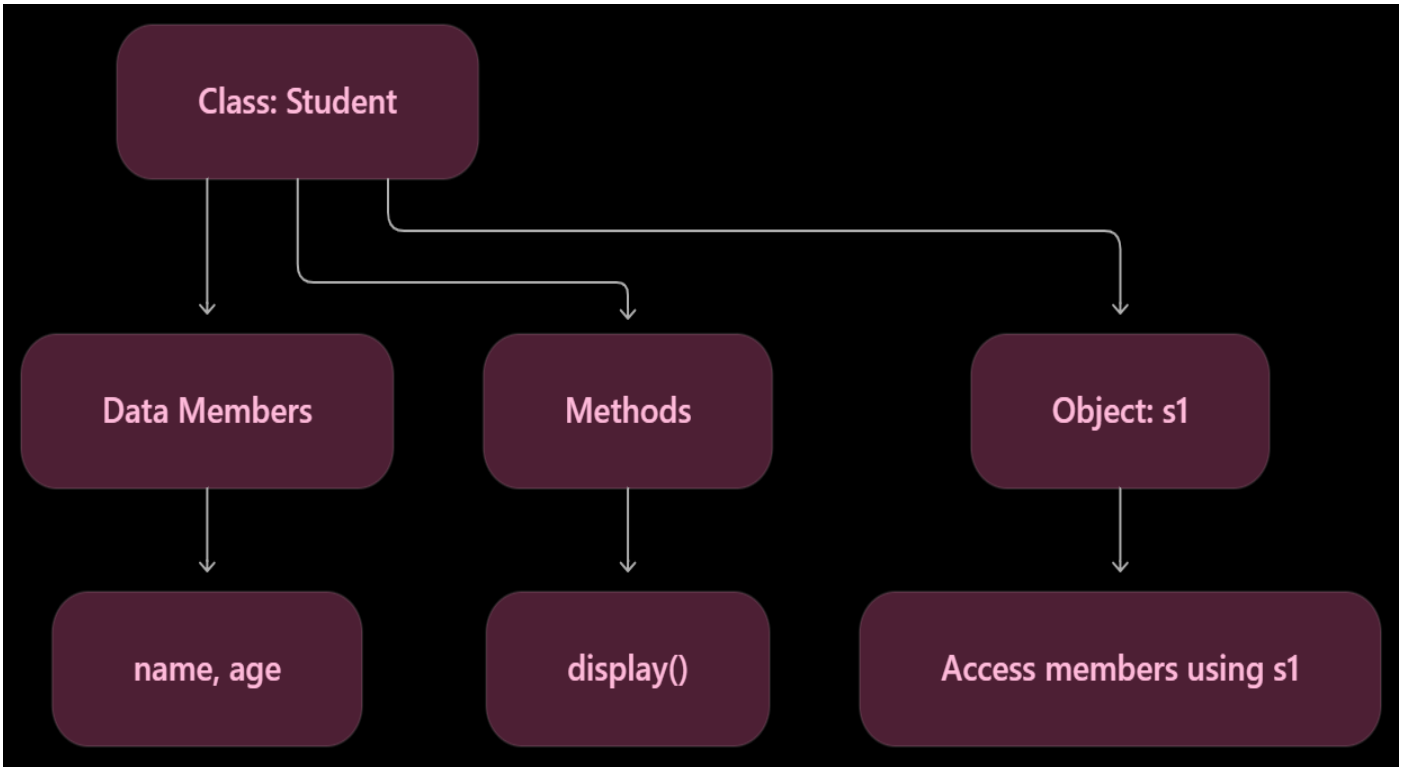
General Form of Object Creation

ClassName objectName = new ClassName();

Explanation:

- ClassName – name of the class.
- objectName – reference variable.
- new – creates an object.
- ClassName() – constructor call.

Diagram



Accessing Data Members and Methods

The dot (.) operator is used to access accessible data members and methods of an object.

Example Program

```
class Student {

  String name;
  int age;

  void display() {
    System.out.println(name);
    System.out.println(age);
  }
}

class Main {
  public static void main(String[] args) {

    Student s1 = new Student();

    s1.name = "Rahul";
    s1.age = 20;

    s1.display();
  }
}
```

Output

Rahul

20

Explanation

1. Student is a class.
2. name and age are data members.

3. display() is a method.
4. new Student() creates an object.
5. s1.name and s1.age access the data members.
6. s1.display() calls the method.

Advantages

- Supports code organization.
- Combines data and methods.
- Helps create multiple objects.
- Supports encapsulation and code reusability.

Conclusion: A class defines the structure of an object, and an object is created using the new keyword. Members are accessed using the dot operator, provided they are accessible.

2. Define Constructor. Explain Different Types of Constructors in Java with Program

Definition

A constructor is a special member of a class used to initialize objects.

Important characteristics

1. Constructor name must be the same as the class name.
2. It has no return type, not even void.
3. It is called automatically when an object is created.
4. Constructors can be overloaded.
5. Constructors are not inherited.

Types of Constructors

1. Default constructor – supplied by the compiler if no constructor is declared.
2. No-argument constructor – explicitly written without parameters.
3. Parameterized constructor – accepts parameters to initialize an object.

General Syntax

```
class ClassName {

  ClassName() {
    // Initialization
  }
```

```
}
```

Program Demonstrating Constructors

```
class Student {

  int age;
  String name;

  // No-argument constructor
  Student() {
    age = 18;
    name = "Unknown";
  }

  // Parameterized constructor
  Student(int a, String n) {
    age = a;
    name = n;
  }

  void display() {
    System.out.println(name + " " + age);
  }
}

class Main {
  public static void main(String[] args) {

    Student s1 = new Student();
    Student s2 = new Student(20, "Rahul");

    s1.display();
```

```
    s2.display();
  }
}
```

Output

Unknown 18

Rahul 20

Explanation

- Student() initializes the object with default values specified by the programmer.
- Student(int a, String n) initializes the object using supplied arguments.
- Both constructors have the same name but different parameter lists. This is constructor overloading.

Advantages

1. Initializes objects automatically.
2. Supports constructor overloading.
3. Improves object creation.
4. Reduces repeated initialization code.

Conclusion: Constructors initialize objects when they are created. Java supports no-argument and parameterized constructors, while the compiler supplies a default constructor when no constructor is declared.

3. What is Instance Variable Hiding? How to Overcome It? Explain the Importance of this Keyword

Definition

Instance variable hiding occurs when a local variable or constructor/method parameter has the same name as an instance variable.

In this situation, the local variable or parameter takes precedence within its scope.

Example of Variable Hiding

```
class Student {

  int age;

  Student(int age) {
```

```
    age = age;
  }
}
```

Here, the parameter age hides the instance variable age.

The statement age = age; assigns the parameter to itself, so the instance variable is not initialized with the supplied value.

How to Overcome Variable Hiding

Java uses the this keyword to refer to the current object.

Correct Program

```
class Student {

  int age;

  Student(int age) {
    this.age = age;
  }

  void display() {
    System.out.println(this.age);
  }
}

class Main {
  public static void main(String[] args) {

    Student s = new Student(20);

    s.display();
  }
}
```

Output



20

Importance of this Keyword

1. Refers to the current object.
2. Distinguishes instance variables from local variables.
3. Calls another constructor of the same class using this().
4. Calls current class methods.
5. Passes the current object as an argument.
6. Returns the current object from a method.

Example: Calling Another Constructor

```
class Student {

  int age;

  Student() {
    this(20);
  }

  Student(int age) {
    this.age = age;
  }
}
```

Here, this(20) calls the parameterized constructor.

Conclusion: The this keyword avoids variable hiding and provides access to the current object's members.

4. Develop a Java Program for Single-Level (Simple) Inheritance

Definition

Single-level inheritance is a type of inheritance in which one subclass inherits directly from one superclass.

Program

```
class Animal {

  void eat() {
    System.out.println("Animal eats");
  }
}

class Dog extends Animal {

  void bark() {
    System.out.println("Dog barks");
  }
}

class Main {
  public static void main(String[] args) {

    Dog d = new Dog();

    d.eat();
    d.bark();
  }
}
```

Output

```
Animal eats
```

Dog barks

Explanation

1. Animal is the superclass.
2. Dog is the subclass.
3. extends establishes inheritance.
4. Dog inherits the accessible eat() method.
5. The object d calls both inherited and child methods.

Advantages

- Code reusability.
- Reduced duplication.
- Easy maintenance.
- Supports method overriding.

Conclusion: Single inheritance allows one class to reuse the accessible properties and methods of another class.

5. What is Inheritance? Identify the Different Types of Inheritance in Java1. Types of Inheritance in Java

Definition

Inheritance is an important feature of Java that allows one class to acquire the properties and methods of another class.

- **The class whose properties are inherited is called the Parent class (Superclass).**
- **The class that inherits properties is called the Child class (Subclass).**
- **Java uses the extends keyword to implement inheritance.**

- Superclass – parent class.
- Subclass – child class.
- extends – keyword used for class inheritance.

Syntax

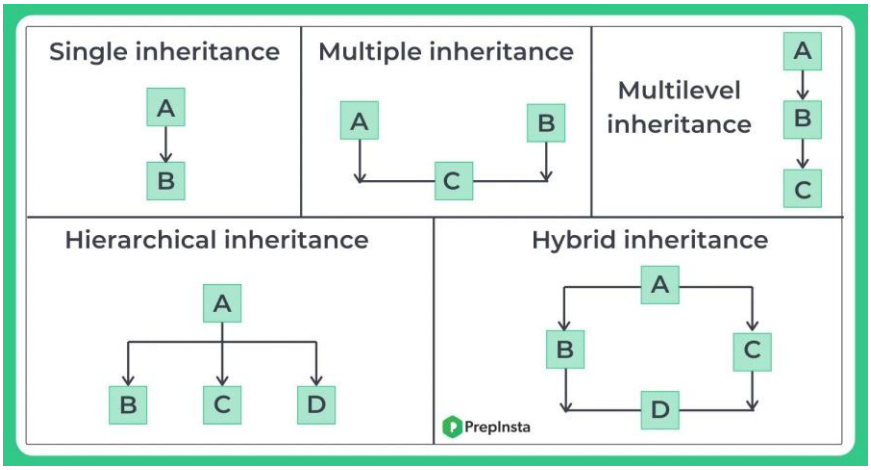
```
class Child extends Parent {
   // Members
}
```

Types of Inheritance

-

Types of Inheritance

Java supports the following types of inheritance:



1. Single Inheritance

One child class inherits properties from one parent class.

```
class Animal {
  void eat() {
    System.out.println("Animal eats");
  }
}

class Dog extends Animal {
  void bark() {
    System.out.println("Dog barks");
  }
}

class Main {
  public static void main(String[] args) {
    Dog d = new Dog();
    d.eat();
    d.bark();
  }
}
```

Output:

Animal eats

Dog barks

2. Multilevel Inheritance

A class inherits from another child class, forming multiple levels.

```
class Animal {
  void eat() {
    System.out.println("Eating");
  }
}

class Dog extends Animal {
  void bark() {
    System.out.println("Barking");
  }
}

class Puppy extends Dog {
  void weep() {
    System.out.println("Weeping");
  }
}

class Main {
  public static void main(String[] args) {
    Puppy p = new Puppy();
    p.eat();
    p.bark();
    p.weep();
  }
```

```
}
```

3. Hierarchical Inheritance

Multiple child classes inherit from a single parent class.

```
class Animal {
  void eat() {
    System.out.println("Eating");
  }
}

class Dog extends Animal {
  void bark() {
    System.out.println("Dog barks");
  }
}

class Cat extends Animal {
  void meow() {
    System.out.println("Cat meows");
  }
}

class Main {
  public static void main(String[] args) {
    Dog d = new Dog();
    Cat c = new Cat();

    d.eat();
    d.bark();

    c.eat();
```

```
    c.meow();
  }
}
```

4. Multiple Inheritance

A class inherits properties from more than one parent class.

Java does not support multiple inheritance through classes to avoid ambiguity. However, it supports multiple inheritance through interfaces.

```
interface A {
  void show();
}

interface B {
  void display();
}

class C implements A, B {
  public void show() {
    System.out.println("Interface A");
  }

  public void display() {
    System.out.println("Interface B");
  }
}

class Main {
  public static void main(String[] args) {
    C obj = new C();
    obj.show();
    obj.display();
```

```
  }
}
```

5. Hybrid Inheritance

Hybrid inheritance is a combination of two or more types of inheritance.

Java can achieve hybrid inheritance using interfaces, but not through multiple inheritance of classes.

Advantages of Inheritance

1. **Code reusability.**
2. **Reduces code duplication.**
3. **Supports method overriding.**
4. **Improves program organization.**
5. **Makes programs easier to maintain.**

Conclusion: Inheritance is used to reuse existing code and establish relationships between classes.

.

6. Develop a Java Program to Demonstrate Multilevel Inheritance

Definition

Multilevel inheritance occurs when a class inherits from another class, and a third class inherits from the second class.

Program

```
class Animal {

  void eat() {
    System.out.println("Animal eats");
  }
}

class Dog extends Animal {

  void bark() {
    System.out.println("Dog barks");
```

```
  }
}

class Puppy extends Dog {

  void weep() {
    System.out.println("Puppy weeps");
  }
}

class Main {
  public static void main(String[] args) {

    Puppy p = new Puppy();

    p.eat();
    p.bark();
    p.weep();
  }
}
```

Output

Animal eats

Dog barks

Puppy weeps

Explanation

1. Animal is the base class.
2. Dog inherits from Animal.
3. Puppy inherits from Dog.
4. Puppy can use accessible methods from both Dog and Animal.
5. The object p calls all three methods.

Advantages



- Reuses methods from multiple levels.
- Reduces code duplication.
- Organizes related classes.
- Supports extension of existing classes.

Conclusion: Multilevel inheritance allows a child class to access accessible members from its parent and ancestors through an inheritance chain.

7. What is Method Overriding? Develop a Java Program

Definition

Method overriding occurs when a subclass provides its own implementation of an inherited instance method with the same signature as the superclass method.

It is used to achieve runtime polymorphism.

Rules

1. The method name must be the same.
2. The parameter list must be the same.
3. The return type must be compatible.
4. The method must be inherited and accessible.
5. The overriding method cannot have more restrictive access.
6. @Override is used to identify overriding.

Program

```
class Animal {

  void sound() {
    System.out.println("Animal makes sound");
  }
}

class Dog extends Animal {

  @Override
  void sound() {
```

```
    System.out.println("Dog barks");
  }
}

class Main {
  public static void main(String[] args) {

    Dog d = new Dog();

    d.sound();
  }
}
```

Output

Dog barks

Explanation

- Animal defines the sound() method.
- Dog overrides the same method.
- When d.sound() is called, the subclass implementation executes.

Advantages

1. Supports runtime polymorphism.
2. Provides specialized behavior.
3. Improves flexibility.
4. Supports extensible program design.

Conclusion: Method overriding allows a subclass to provide a specific implementation of an inherited method.

8. What is Dynamic Method Dispatch? Explain with Example

Definition

Dynamic method dispatch is the process in which the JVM determines at runtime which overridden instance method should execute based on the actual object.

It is also called runtime polymorphism.

Important Concept

A superclass reference variable can refer to a subclass object.

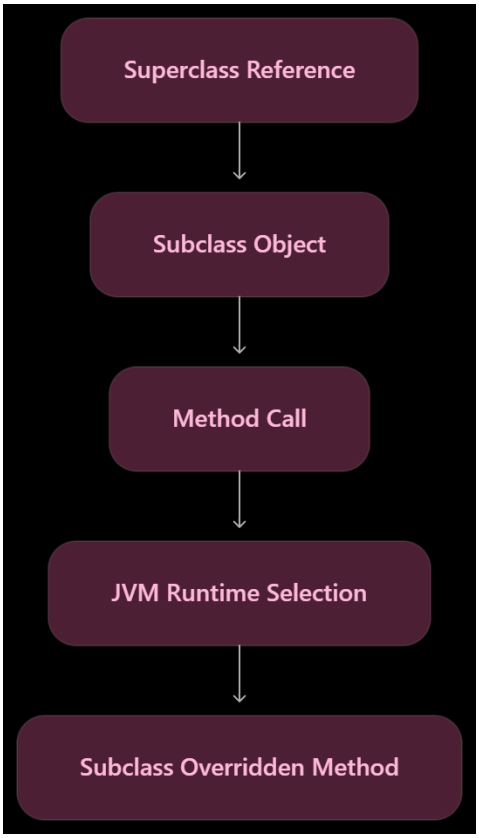
Parent p = new Child();

Here:

- Reference type = Parent
- Actual object type = Child

The overridden method of the actual object is selected at runtime.

Diagram



Program

```
class Animal {

   void sound() {
      System.out.println("Animal sound");
   }
}

class Dog extends Animal {
```

```
  @Override
  void sound() {
    System.out.println("Dog barks");
  }
}

class Main {
  public static void main(String[] args) {

    Animal a = new Dog();

    a.sound();
  }
}
```

Output

Dog barks

Explanation

1. Animal is the superclass.
2. Dog is the subclass.
3. Animal a creates a superclass reference.
4. new Dog() creates a subclass object.
5. a.sound() invokes the overridden method in Dog.

Advantages

- Supports runtime polymorphism.
- Provides flexibility.
- Allows superclass references to work with subclass objects.
- Makes programs extensible.

Conclusion: Dynamic method dispatch selects an overridden method at runtime according to the actual object, not simply the reference variable's declared type.

9. Develop a Java Program to Demonstrate the super Keyword in Inheritance

Definition

The super keyword refers to the immediate superclass object context.

It is used to access superclass members and invoke superclass constructors.

Uses of super

1. Access superclass variables.
2. Call superclass methods.
3. Invoke superclass constructors using super().

Program

```
class Animal {

  int age = 10;

  Animal() {
    System.out.println("Animal constructor");
  }

  void display() {
    System.out.println("Animal method");
  }
}

class Dog extends Animal {

  int age = 5;

  Dog() {
    super();
    System.out.println("Dog constructor");
  }

  void show() {
```

```
        System.out.println(super.age);

        super.display();

        System.out.println(this.age);
    }
}

class Main {
    public static void main(String[] args) {

        Dog d = new Dog();

        d.show();
    }
}
```

Output

```
Animal constructor
Dog constructor
10
Animal method
5
```

Explanation

- super() calls the parent constructor.
- super.age accesses the parent variable.
- super.display() calls the parent method.
- this.age accesses the child variable.

Advantages

1. Accesses superclass members.
2. Resolves variable hiding.

3. Supports constructor chaining.
4. Improves inheritance implementation.

Conclusion: The super keyword is used to access immediate superclass members and constructors in Java inheritance.

10. What is a Package in Java? Explain How to Create and Access a Package

Definition

A package is a collection of related classes, interfaces, and subpackages grouped together under a common name.

Packages are used to organize Java programs and avoid naming conflicts.

Types of Packages

1. Built-in packages – provided by Java.
2. User-defined packages – created by programmers.

Examples of built-in packages:

- java.lang
- java.util
- java.io

Steps to Create a Package

Step 1: Declare the package

```
package mypack;
```

The package declaration must appear at the beginning of the source file, before imports and class declarations.

Step 2: Create a class inside the package

```
package mypack;


public class Student {


  public void display() {
    System.out.println("Welcome to mypack");
  }
}
```

Step 3: Import the package

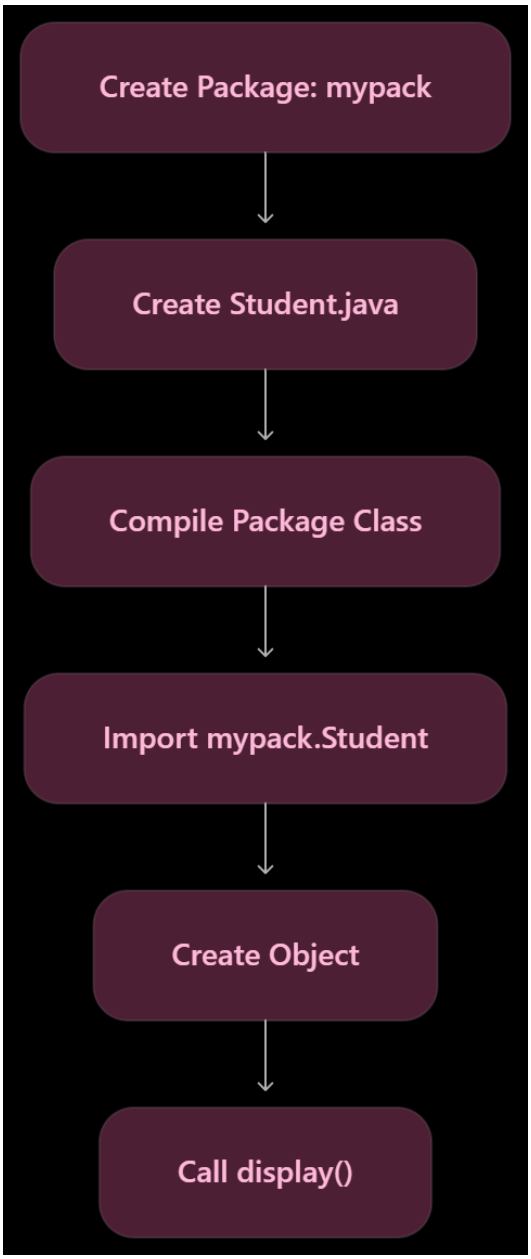
```
import mypack.Student;
```

Step 4: Access the class

Student s = new Student();

s.display();

Diagram



Advantages

1. Organizes related classes.
2. Avoids naming conflicts.
3. Supports access control.
4. Improves code reusability.
5. Makes programs easier to maintain.
6.

11. Develop a Java Program to Create a Package Named mypack and Import It

Step 1: Create Package Class

Create a file named Message.java.

```
package mypack;


public class Message {


  public void display() {
    System.out.println("Welcome to mypack package");
  }
}
```

Step 2: Create Main Class

Create a file named Main.java.

```
import mypack.Message;


class Main {


  public static void main(String[] args) {


    Message m = new Message();


    m.display();
  }
}
```

Step 3: Compile the Package

Open the terminal in the project directory.

javac -d . Message.java

The -d . option tells the compiler to place the class file in the appropriate package directory under the current directory.

Step 4: Compile the Main Class

javac Main.java



Step 5: Run

java Main

Output

Welcome to mypack package

Explanation

1. mypack is a user-defined package.
2. Message is a public class inside the package.
3. import mypack.Message imports the class.
4. new Message() creates an object.
5. m.display() calls the method.

Conclusion: User-defined packages organize Java classes and allow them to be imported and used in other classes.

12. Write Short Notes on (i) Access Specifiers in Package (ii) Variables in Interfaces

i) Access Specifiers in Packages

Access specifiers control the visibility of class members within and outside packages.

Specifier	Same Class	Same Package	Other Package Subclass	Other Package
private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes*	No
public	Yes	Yes	Yes	Yes

Explanation

- private: Accessible only within the declaring class.
- default: Accessible within the same package.
- protected: Accessible within the same package and through inheritance in other packages, subject to Java's access rules.
- public: Accessible wherever the containing class is accessible.

ii) Variables in Interfaces

Definition

Variables declared in a Java interface are implicitly:

- public

- static
- final

Therefore, interface variables are constants.

Example

```
interface Student {

  int MAX_MARKS = 100;
}

class Main {

  public static void main(String[] args) {

    System.out.println(Student.MAX_MARKS);
  }
}
```

Output

100

Important Points

1. Interface variables are public constants.
2. They must be initialized when declared.
3. They cannot be modified after initialization.
4. They can be accessed using the interface name.
5. Implementing classes do not need to declare the constant again.

Conclusion: Access specifiers control visibility in packages, while interface variables are implicitly public, static, and final.

13. Define Interface. Develop a Program Showing How a Class Implements an Interface

Definition

An interface is a reference type in Java that defines a contract that implementing classes must follow.

It is used to achieve abstraction and multiple inheritance of type.

Syntax

```
interface InterfaceName {

  void methodName();
}
```

Implementing an Interface

A class uses the implements keyword to implement an interface.

Program

```
interface Animal {

  void sound();
}

class Dog implements Animal {

  public void sound() {
    System.out.println("Dog barks");
  }
}

class Main {

  public static void main(String[] args) {

    Dog d = new Dog();

    d.sound();
  }
}
```

Output

```
Dog barks
```



Explanation

1. Animal is an interface.
2. sound() is an abstract interface method.
3. Dog implements Animal.
4. Dog provides the implementation of sound().
5. The object calls the implemented method.

Advantages

1. Supports abstraction.
2. Supports multiple inheritance of type.
3. Provides a common contract.
4. Improves code flexibility.
5. Supports loose coupling.

Conclusion: Interfaces define a contract, and implementing classes provide the method implementations required by that contract.

14. Summarize (1) final Keyword (2) Garbage Collection

i) final Keyword

Definition

The final keyword is used to restrict modification of variables, methods, and classes.

Uses of final

1. final variable

A final variable can be assigned only once.

```
final int MAX = 100;
```

2. final method

A final method cannot be overridden by a subclass.

```
class A {

  final void display() {
    System.out.println("Final method");
  }
}
```

3. final class

A final class cannot be extended.

final class A {

}

Advantages

- Prevents unwanted modification.
- Prevents method overriding.
- Prevents inheritance of final classes.
- Supports constant declaration.

ii) Garbage Collection

Definition

Garbage collection is the automatic process of identifying and reclaiming heap memory occupied by objects that are no longer reachable by a program.

Java manages object memory through the JVM garbage collector.

Working Steps

1. Objects are created in heap memory.
2. References are used to access objects.
3. When an object becomes unreachable, it becomes eligible for garbage collection.
4. The JVM garbage collector may reclaim its memory.
5. Memory becomes available for future allocations.

Example

```
class Main {

  public static void main(String[] args) {

    Student s = new Student();

    s = null;
  }
}
```

```
class Student {

}
```

Here, the created Student object becomes eligible for garbage collection after s is assigned null, assuming no other references exist.

Important Points

- Garbage collection is automatic.
- The JVM decides when to perform it.
- System.gc() only requests garbage collection; it does not guarantee immediate execution.
- Garbage collection manages memory, not every external resource.

Advantages

1. Automatic memory management.
2. Reduces manual memory deallocation.
3. Helps prevent memory errors caused by incorrect manual freeing.
4. Improves memory utilization.

Disadvantages

1. Garbage collection timing is unpredictable.
2. May introduce performance overhead.
3. Does not automatically close files or other external resources.

Conclusion: The final keyword restricts modification and inheritance, while garbage collection automatically reclaims heap memory from unreachable objects.

15. Develop a Java Program to Demonstrate a Nested Interface

Definition

A nested interface is an interface declared inside another class or interface.

It is used to group related interface definitions and improve program organization.

Syntax

```
class Outer {


  interface Inner {
    void display();
  }
}
```

A nested interface declared inside a class is implicitly static.

Java Program

```
class Outer {

  interface Inner {

    void display();
  }
}

class Demo implements Outer.Inner {

  public void display() {
    System.out.println("Nested interface implemented");
  }
}

class Main {

  public static void main(String[] args) {

    Demo d = new Demo();

    d.display();
  }
}
```

Output

Nested interface implemented

Explanation

1. Outer is the outer class.
2. Inner is an interface declared inside Outer.



3. Demo implements the nested interface using Outer.Inner.
4. Demo provides the implementation of display().
5. The object calls the implemented method.

Advantages

1. Groups related interfaces.
2. Improves code organization.
3. Avoids unnecessary top-level interface declarations.
4. Supports abstraction.
5. Helps create structured programs.

Conclusion: Nested interfaces allow interfaces to be declared inside classes or other interfaces. They improve organization and can be implemented using the qualified interface name.

MODULE 3

Java Programming – Exception Handling

B.Tech 3rd Semester | Exam Answers

These answers are written in simple exam language, with definitions, important points, Java programs, and outputs. Suitable for preparing 8–10 mark answers.

1. What is an exception? Identify the different types of exceptions.

Definition

An exception is an abnormal condition or error that occurs during the execution of a Java program and interrupts its normal flow.

Java provides an exception-handling mechanism to detect and handle these errors so that the program can continue executing whenever possible.

Example

```
class Main {
  public static void main(String[] args) {
    int a = 10;
    int b = 0;
```

```
        int c = a / b;

        System.out.println(c);
    }
}
```

Output:

Exception in thread "main" java.lang.ArithmeticException

Here, division by zero causes an exception.

Types of exceptions in Java

Java exceptions are commonly classified into two types:

Exceptions

Checked Exceptions

Checked by compiler

IOException, SQLException

Unchecked Exceptions

Occur at runtime

ArithmeticException, NullPointerException

1. Checked exceptions

- Checked by the compiler at compile time.
- Must be handled using try-catch or declared using throws.
- Usually occur due to external operations such as file handling.

Examples:

- IOException
- SQLException
- ClassNotFoundException
- FileNotFoundException

2. Unchecked exceptions

- Occur during program execution.
- Compiler does not force the programmer to handle them.
- Generally caused by programming mistakes.

Examples:

- ArithmeticException
- NullPointerException
- ArrayIndexOutOfBoundsException
- NumberFormatException

3. Errors

Errors are serious problems generally caused by the JVM or system environment. They are not normally handled by application programs.

Examples:

- OutOfMemoryError
- StackOverflowError

Conclusion

Exceptions help identify abnormal conditions in a program. Java supports checked exceptions, unchecked exceptions, and errors under its exception hierarchy.

2. What is exception handling? Discuss the mechanism to handle exceptions.

Definition

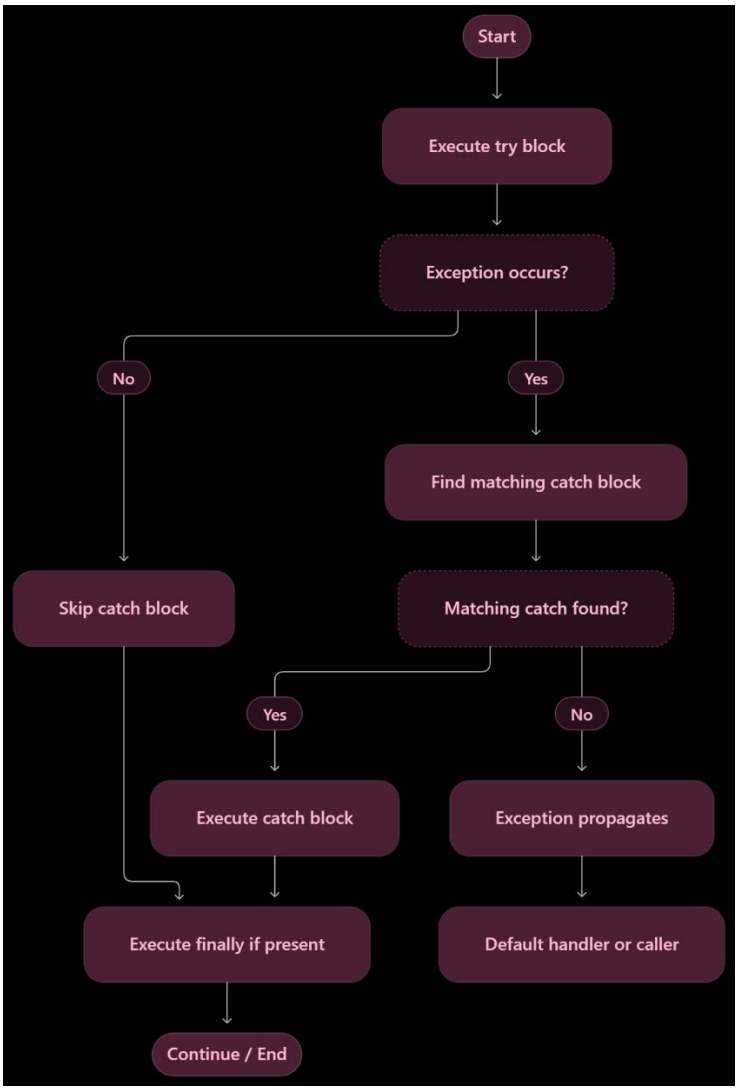
Exception handling is a mechanism in Java used to detect, handle, and manage runtime errors so that the normal flow of the program can be maintained.

Java provides the following keywords:

Keyword	Purpose
try	Contains code that may produce an exception
catch	Handles the exception
finally	Contains cleanup code

throw	Explicitly throws an exception
throws	Declares exceptions that a method may throw

Mechanism of exception handling



Steps

1. The JVM executes the statements inside the try block.
2. If no exception occurs, the catch block is skipped.
3. If an exception occurs, the remaining statements in the try block are skipped.
4. JVM searches for a matching catch block.
5. If a matching handler is found, it executes.
6. The finally block executes if present, normally whether an exception occurred or not.

7. If no matching handler is found, the exception propagates to the caller or JVM's default handler.

Java program

```
class ExceptionDemo {
   public static void main(String[] args) {

      int a = 10;
      int b = 0;

      try {
         int result = a / b;
         System.out.println("Result: " + result);
      }
      catch (ArithmeticException e) {
         System.out.println("Cannot divide by zero");
      }
      finally {
         System.out.println("Finally block executed");
      }

      System.out.println("Program continues");
   }
}
```

Output:

Cannot divide by zero

Finally block executed

Program continues

Advantages



1. Prevents abrupt program termination when exceptions are handled.
2. Separates error-handling code from normal code.
3. Improves program reliability.
4. Helps identify errors using exception messages.
5. Allows resources to be cleaned up.

3. Apply nested try statements in Java and demonstrate with an example.

Definition

A nested try statement is a try block placed inside another try block.

It is used when different sections of a program may produce different exceptions and require separate handling.

Syntax

```
try {
   // Outer try block


   try {
      // Inner try block
   }
   catch (ExceptionType1 e) {
      // Inner catch
   }


}
catch (ExceptionType2 e) {
   // Outer catch
}
```

Java program

```
class NestedTryDemo {
   public static void main(String[] args) {
```

```
        try {
            System.out.println("Outer try block");

            int a = 10;
            int b = 2;

            System.out.println("Division: " + (a / b));

            try {
                int arr[] = {10, 20, 30};

                System.out.println(arr[5]);
            }
            catch (ArrayIndexOutOfBoundsException e) {
                System.out.println("Array index is invalid");
            }

        }
        catch (ArithmeticException e) {
            System.out.println("Arithmetic error");
        }

        System.out.println("Program continues");
    }
}
```

Output:



Outer try block

Division: 5

Array index is invalid

Program continues

Explanation

1. The outer try block executes first.
2. Division of 10 by 2 is successful.
3. The inner try block attempts to access array index 5.
4. The array contains only three elements, so an exception occurs.
5. The inner catch handles the exception.
6. The program continues after the nested structure.

Advantages

- Handles different exceptions separately.
- Provides better control over program execution.
- Makes complex exception handling easier.
- Allows inner and outer blocks to handle different errors.

4. Develop a Java program to demonstrate multiple catch blocks.

Definition

Multiple catch blocks are used with a single try block to handle different types of exceptions separately.

Each catch block handles a particular exception type.

Syntax

```
try {
   // Statements
}
catch (ExceptionType1 e) {
   // Handle first exception
}
```

```
catch (ExceptionType2 e) {
   // Handle second exception
}
```

Java program

```
class MultipleCatchDemo {
   public static void main(String[] args) {

      try {
         int a = 10;
         int b = 0;

         int result = a / b;

         System.out.println(result);

         int arr[] = {10, 20, 30};

         System.out.println(arr[5]);
      }
      catch (ArithmeticException e) {
         System.out.println("Arithmetic exception occurred");
      }
      catch (ArrayIndexOutOfBoundsException e) {
         System.out.println("Array index exception occurred");
      }
      catch (Exception e) {
         System.out.println("General exception occurred");
```

```
    }

    System.out.println("Program continues");
  }
}
```

Output:

Arithmetic exception occurred

Program continues

Explanation

- Division by zero causes ArithmeticException.
- The first matching catch block executes.
- The remaining statements inside the try block are skipped.
- The array statement is not executed in this run.
- The final statement outside the try-catch executes.

Important points

1. One try block can have multiple catch blocks.
2. Only the first matching catch block executes for a particular exception.
3. Specific exceptions should be written before general exceptions.
4. Exception should normally be placed last because it is the superclass of most exception classes.

5. Discuss the following: (1) throw (2) throws (3) finally

1. throw keyword

The throw keyword is used to explicitly generate an exception.

It is followed by an exception object.

Syntax

throw new ExceptionType("message");

Example

throw new ArithmeticException("Invalid operation");

Important points

- Used inside a method or block.
- Throws one exception object at a time.
- Transfers control to the appropriate exception handler.
- Can be used for built-in or user-defined exceptions.

2. throws keyword

The throws keyword is used in a method declaration to indicate that the method may generate one or more exceptions.

It informs the caller that the exception may need to be handled.

Syntax

```
returnType methodName() throws ExceptionType {
   // Statements
}
```

Example

```
static void display() throws IOException {
   // Code that may cause IOException
}
```

Important points

- Written in the method declaration.
- Can declare multiple exception types separated by commas.
- Commonly used with checked exceptions.
- The caller must handle or further declare checked exceptions.

3. finally block

The finally block contains statements that are normally executed after the try and catch blocks.

It is mainly used for cleanup operations.

Syntax

```
try {
```



```
    // Statements
}
catch (Exception e) {
    // Handle exception
}
finally {
    // Cleanup statements
}
```

Example

```
class FinallyDemo {
    public static void main(String[] args) {

        try {
            int a = 10 / 2;
            System.out.println(a);
        }
        catch (Exception e) {
            System.out.println("Exception occurred");
        }
        finally {
            System.out.println("Finally executed");
        }
    }
}
```

Output:

```
5
Finally executed
```

Difference between throw, throws, and finally

throw	throws	finally
Explicitly throws an exception	Declares possible exceptions	Executes cleanup code
Used inside method/block	Used in method declaration	Used after try/catch
Throws an exception object	Lists exception types	Does not throw by itself
Followed by an object	Followed by exception class names	Followed by a block
Example: throw new IOException()	Example: throws IOException	Example: finally { }

6. Develop a Java program to demonstrate the use of throw.

Definition

The throw keyword is used to explicitly raise an exception when a particular condition occurs.

Java program

```
class ThrowDemo {
  public static void main(String[] args) {

    int age = 15;

    try {
      if (age < 18) {
        throw new ArithmeticException(
          "Not eligible"
        );
      }
```

```
            System.out.println("Eligible");
        }
        catch (ArithmeticException e) {
            System.out.println("Exception: " + e.getMessage());
        }

        System.out.println("Program continues");
    }
}
```

Output:

Exception: Not eligible

Program continues

Explanation

1. The variable age contains 15.
2. The condition age < 18 is true.
3. The throw keyword creates and throws an ArithmeticException.
4. The catch block handles the exception.
5. The program continues after exception handling.

Conclusion

The throw keyword is useful for generating exceptions explicitly when program conditions are not satisfied.

7. Build a Java program to demonstrate the use of throws.

Definition

The throws keyword declares exceptions that may occur inside a method.

It transfers the responsibility of handling a checked exception to the calling method.

Java program

```
import java.io.IOException;
```

```
class ThrowsDemo {

   static void display() throws IOException {
      throw new IOException("Input output error");
   }

   public static void main(String[] args) {

      try {
         display();
      }
      catch (IOException e) {
         System.out.println("Exception handled");
         System.out.println(e.getMessage());
      }

      System.out.println("Program continues");
   }
}
```

Output:

Exception handled

Input output error

Program continues

Explanation

1. The display() method declares IOException using throws.
2. The method throws an IOException.
3. The main() method calls display().

4. The exception is handled using try-catch.
5. The program continues after handling the exception.

Important points

- throws is written in the method declaration.
- It does not itself generate an exception.
- It can declare more than one exception.
- It is especially useful for checked exceptions.

8. Develop a program for nested try blocks.

This question is similar to Question 3. Here is another simple program demonstrating nested try blocks with two different exceptions.

Java program

```
class NestedTry {
  public static void main(String[] args) {

    try {
      int a = 20;
      int b = 0;

      try {
        int result = a / b;
        System.out.println(result);
      }
      catch (ArithmeticException e) {
        System.out.println("Division by zero");
      }

      try {
        int arr[] = {1, 2, 3};
```

```
            System.out.println(arr[10]);
        }
        catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Invalid array index");
        }

    }
    catch (Exception e) {
        System.out.println("Outer exception");
    }

    System.out.println("End of program");
  }
}
```

Output:

Division by zero

Invalid array index

End of program

Explanation

- The outer try contains two inner try blocks.
- The first inner block handles division by zero.
- The second inner block handles invalid array access.
- Each inner exception is handled separately.
- The outer catch handles any other exception not handled inside.

Advantages

1. Handles different exceptions in separate blocks.

2. Improves readability.
3. Gives better control over exception handling.
4. Helps prevent abnormal termination.

9. Develop a program to raise a custom exception/user-defined exception for DivisionByZero using try and catch blocks.

Definition

A user-defined exception is an exception created by the programmer to handle application-specific errors.

In Java, a custom exception can be created by extending:

- Exception – creates a checked exception.
- RuntimeException – creates an unchecked exception.

Here, we create a custom exception named DivisionByZeroException.

Java program

```
class DivisionByZeroException extends Exception {

  DivisionByZeroException(String message) {
    super(message);
  }
}

class CustomExceptionDemo {

  public static void main(String[] args) {

    int a = 10;
    int b = 0;

    try {
```

```
        if (b == 0) {
            throw new DivisionByZeroException(
                "Division by zero is not allowed"
            );
        }

        int result = a / b;

        System.out.println("Result: " + result);
    }
    catch (DivisionByZeroException e) {
        System.out.println("Custom Exception: "
                + e.getMessage());
    }

    System.out.println("Program continues");
  }
}
```

Output:

Custom Exception: Division by zero is not allowed

Program continues

Explanation

1. DivisionByZeroException is created by extending the Exception class.
2. Its constructor accepts an error message.
3. The program checks whether the denominator is zero.
4. If b == 0, the throw keyword raises the custom exception.
5. The catch block handles the exception.



6. The program continues after exception handling.

Advantages of user-defined exceptions

1. Handles application-specific errors.
2. Makes error messages meaningful.
3. Improves program readability.
4. Separates error conditions from normal execution.
5. Provides better control over exception handling.